

PARQUE CIENTÍFICO Y TECNOLÓGICO UTPL

ISBEN Solutions – Plataforma Digital: ConectaMarket

Versión 1.0.0

Manual de usuario

Autores:
Líder de grupo: Byron Alvarez
Estudiante: Cody Cabrera
Estudiante: Byron Alvarez
Docente Tutor: Ing. René Elizalde

1. Capítulo 1: Generalidades del Proyecto

1.1. Introducción

El presente documento detalla el análisis y planificación para el desarrollo de una plataforma web de gestión de pedidos orientada a la comercialización de productos e insumos para micromercados. El sistema actúa como un puente tecnológico entre empresas mayoristas y tiendas locales (principalmente en zonas rurales), brindando oportunidades a vendedores independientes mediante un catálogo con inventario en tiempo real, incentivos por gamificación y un stack tecnológico moderno desarrollado de forma Full-Stack utilizando el framework web de Django con PostgreSQL para la persistencia de datos.

1.2. Problemática

Las empresas mayoristas y proveedores de micromercados en zonas rurales enfrentan una pérdida constante de ventas y clientes debido a un control deficiente y desfasado del inventario entre el punto de abastecimiento y la toma de pedidos en campo. Simultáneamente, la adopción de soluciones de compra directa (e-commerce) es inviable debido a la barrera tecnológica y desconfianza de los dueños de tiendas (población mayor a 60 años). Esto hace indispensable la figura de vendedores intermediarios, quienes, al no contar con información de stock en tiempo real ni incentivos dinámicos, generan incumplimientos en las entregas y estancamiento de productos.

1.3. Metodología o estrategia de desarrollo

Se empleará un enfoque ágil basado en Scrum, adaptado para el desarrollo iterativo e incremental del software. Esta metodología permitirá entregar módulos funcionales en ciclos cortos. La comunicación del equipo se gestionará mediante juntas de planificación de sprint, organizaciones diarias de sincronización (daily stand-ups) y juntas de revisión al finalizar cada iteración. Se utilizarán herramientas de control de versiones (Git) para el seguimiento de las tareas.

2. Capítulo 2: Planificación y Análisis

2.1. Análisis

El proyecto se enfoca en resolver la brecha de información entre el inventario de la comercializadora y el vendedor en campo. Los interesados principales incluyen a las Comercializadoras (quienes pagan la suscripción y proveen el inventario), los Vendedores (usuarios que gestionan rutas, registran pedidos y ganan comisiones/puntos), y los dueños de Micromercados, quienes ahora tienen la capacidad de interactuar directamente con la plataforma mediante el rol de Tienda para auto-gestionar pedidos y

llevar control de su inventario local. El alcance se centra en la intermediación, toma de pedidos y gestión de inventarios, dejando fuera la logística de entrega física.

2.2. Requisitos funcionales

ID	Descripción
RF-01	El sistema debe permitir el registro y autenticación de usuarios en la plataforma bajo los roles de Comercializadora, Vendedor y Tienda.
RF-02	La plataforma debe permitir a la Comercializadora gestionar su catálogo completo, visualizar indicadores clave de rendimiento (KPIs) en tiempo real y configurar sus políticas de venta (porcentaje de comisión para vendedores).
RF-03	El sistema debe permitir a Vendedores y Tiendas visualizar el catálogo disponible y registrar pedidos verificando disponibilidad en tiempo real.
RF-04	El sistema debe calcular las comisiones de la plataforma y procesar la gamificación al vendedor de manera automática, basándose en el porcentaje personalizado definido por cada Comercializadora.
RF-05	El sistema debe gestionar una bandeja compartida de pedidos: los creados directamente por las Tiendas quedarán disponibles para ser asignados al primer Vendedor que los procese.
RF-06	El sistema debe actualizar el inventario propio de una Tienda (InventarioTienda) automáticamente al entregar pedidos, y permitir al dueño del micromercado realizar ajustes físicos (CRUD) sobre su stock en percha.

2.3. Requisitos no funcionales

ID	Descripción
RNF-01	Arquitectura: La plataforma debe estar construida de forma monolítica (Full-Stack) utilizando el sistema de plantillas de Django, delegando la lógica de negocio a los modelos (Fat Models, Skinny Views).
RNF-02	Base de Datos: Se utilizará PostgreSQL para garantizar la integridad, implementando bloqueo optimista (Optimistic Locking) para prevenir inconsistencias en el inventario.
RNF-03	Usabilidad: La interfaz renderizada por Django debe ser completamente responsiva (Mobile-First) con soporte nativo de CSS para su correcta visualización en campo.

2.4. Historias de usuario / Casos de Uso

- HU-01 (Vendedor): Como vendedor, quiero ver el inventario actualizado en tiempo real para evitar vender productos que están fuera de stock y perder credibilidad con las tiendas.
- HU-02 (Comercializadora): Como administrador de la comercializadora, quiero visualizar en mi panel de control indicadores clave (KPIs) como cuentas por cobrar, tiendas abastecidas y alertas de stock, para tomar decisiones de negocio rápidamente.
- HU-03 (Vendedor): Como vendedor, quiero visualizar mis puntos acumulados en un dashboard para canjearlos y monitorear mis ganancias por comisiones.
- HU-04 (Tienda): Como dueño de micromercado, quiero poder ingresar al sistema y generar pedidos directamente de los catálogos para abastecerme sin esperar la visita física de un vendedor.
- HU-05 (Tienda): Como dueño de micromercado, quiero consultar y ajustar manualmente el estado de mis inventarios locales (InventarioTienda) para mantener sincronizado mi stock físico frente a mermas o ventas al consumidor final.

- HU-06 (Comercializadora): Como administrador de la comercializadora, quiero poder configurar el porcentaje de comisión que ofrezco a los vendedores, para tener flexibilidad financiera según mi margen de ganancia.

2.5. Sprint

El desarrollo se dividirá en Sprints de 2 semanas:

1. Sprint 1: Diseño de base de datos en PostgreSQL, configuración de Django y creación del módulo de Autenticación/Roles (Vendedor, Comercio, Tienda).
2. Sprint 2: Desarrollo del CRUD de productos, categorías, inventarios e implementación de las interfaces de usuario con el motor de plantillas de Django.
3. Sprint 3: Módulo de toma de pedidos, lógica transaccional, cálculos automáticos de montos y manejo de bandeja compartida para tiendas.
4. Sprint 4: Implementación del motor de gamificación (Campañas Recompensa), liquidaciones y tableros (dashboards) finales para cada rol.

2.6. Diagramas: Clases y BD (diseño lógico) - Preguntas y respuesta al diagrama

El diseño lógico en PostgreSQL contempla las siguientes entidades principales: Usuario, Vendedor, Comercializadora, Tienda, Categoría, Producto, Inventario, InventarioTienda, Pedido, DetallePedido, LiquidacionComercializadora, CampanaRecompensa, y TransaccionPuntos.

3. Diccionario de Datos (Atributos y Reglas de Negocio)

A continuación se detalla el diccionario de datos derivado del diagrama de clases, especificando los tipos de datos a implementar en el ORM de Django y las reglas de negocio transaccionales para PostgreSQL.

Entidad	Atributo	Tipo (Django)	Descripción / Regla de Negocio
Usuario	id	AutoField	Identificador único (PK).
Usuario	ruc	CharField(13)	Registro Único de Contribuyentes. Debe ser unique=True y tener exactamente 13 dígitos.
Usuario	nombres / apellidos	CharField(100)	Nombres y apellidos del usuario.
Usuario	email	EmailField	Correo electrónico. Debe ser unique=True .
Usuario	password	CharField(128)	Contraseña encriptada (Hash PBKDF2).
Usuario	estado_cuenta	BooleanField	Indica si el usuario está activo (default=True).
Usuario	rol	CharField(20)	Opciones (Choices): 'VENDEDOR', 'COMERCIALIZADORA' o 'TIENDA'.
Vendedor	usuario_id	OneToOneField	PK y FK hacia Usuario. Relación 1 a 1.
Vendedor	zona_asignada	CharField(100)	Región o ruta rural de cobertura.
Vendedor	vehiculo_placa	CharField(10)	Placa del vehículo de transporte.
Vendedor	puntos_acumulados	IntegerField	Total de puntos de gamificación. Regla: ≥ 0 .
Vendedor	presupuesto_ventas	DecimalField	Presupuesto asignado para compras al por mayor. Regla: ≥ 0.00 .
Continúa en la siguiente página...			

Entidad	Atributo	Tipo (Django)	Descripción / Regla de Negocio
Comercializadora	usuario_id	OneToOneField	PK y FK hacia Usuario. Relación 1 a 1.
Comercializadora	razon_social	CharField(150)	Nombre legal de la empresa.
Comercializadora	nombre_empresa	CharField(150)	Nombre comercial visible en el catálogo.
Comercializadora	direccion_matriz	TextField	Ubicación principal del negocio.
Comercializadora	suscripcion_activa	BooleanField	Define si sus productos se muestran en el catálogo.
Comercializadora	porcentaje_comision	DecimalField	Porcentaje de comisión asignado a los vendedores (Ej: 10.00). Regla: ≥ 0 .
Tienda	id	AutoField	Identificador único (PK).
Tienda	usuario_id	OneToOneField	PK y FK hacia Usuario. Relación 1 a 1 para propietarios de micromercados.
Tienda	nombre	CharField(100)	Nombre del micromercado.
Tienda	direccion	TextField	Dirección física en la zona rural.
Tienda	telefono	CharField(15)	Teléfono de contacto del micromercado.
Tienda	latitud / longitud	DecimalField	Coordenadas exactas para ubicar la tienda en el mapa.
Categoria	id	AutoField	Identificador único (PK).
Categoria	nombre	CharField(50)	Nombre o clasificación general de familia de productos.
Producto	id	AutoField	Identificador único (PK).
Producto	comercializadora_id	ForeignKey	FK hacia Comercializadora.
Producto	codigo	CharField(50)	Código interno del producto. Debe ser unique=True .
Producto	nombre	CharField(150)	Nombre descriptivo del producto.
Producto	categoria_id	OneToOneField	FK hacia Categoria.
Producto	precio	DecimalField	Costo base de venta al por mayor.
Inventario	id	AutoField	Identificador único (PK).
Inventario	producto_id	OneToOneField	FK hacia Producto (1 a 1 por bodega principal).
Inventario	almacen_origen	CharField(100)	Nombre o código de la bodega.
Inventario	cantidad_disp	PositiveIntegerField	Stock en tiempo real. Regla: No puede ser menor a 0.
Inventario	fecha_actualizacion	DateTimeField	Timestamp de actualización de stock.
Inventario	version	IntegerField	Control de concurrencia (Optimistic Locking).
InventarioTienda	id	AutoField	Identificador único (PK).
InventarioTienda	tienda_id	ForeignKey	FK hacia Tienda a la que pertenece el stock.
InventarioTienda	producto_id	ForeignKey	FK hacia Producto adquirido.
InventarioTienda	cantidad_disp	PositiveIntegerField	Unidades disponibles en la percha local de la tienda.
Pedido	id	AutoField	Identificador único (PK).
Pedido	vendedor_id	ForeignKey	FK hacia Vendedor. Si es null, ingresa a bandeja compartida.
Pedido	tienda_id	ForeignKey	FK hacia Tienda que recibirá los productos.
Pedido	fecha	DateTimeField	Fecha y hora exacta de creación (auto_now_add=True).

Continúa en la siguiente página...

Entidad	Atributo	Tipo (Django)	Descripción / Regla de Negocio
Pedido	estado	CharField(20)	Choices: 'PENDIENTE', 'CONFIRMADO', 'ENTREGADO', 'CANCELADO'.
Pedido	monto_total_tienda	DecimalField	Suma de subtotales. Total a cancelar.
DetallePedido	id	AutoField	Identificador único (PK).
DetallePedido	pedido_id	ForeignKey	FK hacia Pedido. Borrado en cascada.
DetallePedido	producto_id	ForeignKey	FK hacia Producto.
DetallePedido	cantidad	PositiveIntegerField	Unidades solicitadas. Regla: > 0.
DetallePedido	precio_unitario	DecimalField	Precio congelado al momento de la venta.
DetallePedido	subtotal	DecimalField	$cantidad * precio_unitario$.
Liquidacion-Comercializadora	id	AutoField	Identificador único (PK).
Liquidacion-Comercializadora	pedido_id	OneToOneField	FK hacia Pedido. Generado desde la clase modelo de Pedido.
Liquidacion-Comercializadora	monto_comision	DecimalField	Fee del vendedor (Calculado dinámicamente según el porcentaje_comision de la Comercializadora).
Liquidacion-Comercializadora	monto_cobrar	DecimalField	Lo que recibe la comercializadora (Total - Comisión).
Liquidacion-Comercializadora	fecha_liquidacion	DateTimeField	Fecha en que se procesó el corte contable.
Liquidacion-Comercializadora	estado_pago	CharField(20)	Choices: 'PENDIENTE', 'PAGADO_COMERCIALIZADORA'.
Campana-Recompensa	id	AutoField	Identificador único (PK).
Campana-Recompensa	producto_id	ForeignKey	FK hacia Producto.
Campana-Recompensa	nombre_campana	CharField(100)	Identificador de la estrategia (Ej: 'Impulso Lácteos').
Campana-Recompensa	factor_puntos	PositiveIntegerField	Cantidad de puntos extra que da este producto al venderse.
Campana-Recompensa	fecha_inicio	DateTimeField	Inicio de vigencia de la campaña.
Campana-Recompensa	fecha_fin	DateTimeField	Fin de vigencia de la campaña.
Campana-Recompensa	estado	BooleanField	Activa/Inactiva.
Transaccion-Puntos	id	AutoField	Identificador único (PK).
Transaccion-Puntos	vendedor_id	ForeignKey	FK hacia Vendedor.
Transaccion-Puntos	pedido_id	ForeignKey	FK hacia Pedido. Puede ser <code>null=True</code> si es un canje.
Transaccion-Puntos	tipo_transaccion	CharField(15)	Choices: 'INGRESO' (Venta), 'EGRESO' (Canje de apuestas).
Transaccion-Puntos	puntos_ganados	IntegerField	Valor absoluto de la transacción.
Transaccion-Puntos	fecha	DateTimeField	Timestamp de la transacción.

A continuación, se plantean preguntas críticas para validar la robustez del modelo de clases y los requerimientos funcionales actuales:

Pregunta 1 (Concurrencia): ¿Cómo resuelve el modelo de clases la concurrencia si dos vendedores intentan registrar un pedido para el mismo producto simultáneamente, y el stock disponible es insuficiente para ambos?

Respuesta: El modelo en Django centraliza la lógica dentro de una transacción atómica mediante un control de bloqueo optimista utilizando el campo `version` en la entidad `Inventario`. Antes de guardar el `DetallePedido`, la clase del modelo evalúa la disponibilidad y detiene la petición levantando una alerta `StockInsuficienteError` si no existe stock real.

Pregunta 2 (Acoplamiento de Pagos): Si el modelo de negocio permite comisiones variables por venta, ¿dónde se registra esta obligación y cómo se calcula?

Respuesta: La clase `Pedido` posee la función interna `generar_liquidacion()`. Esta función rastrea la Comercializadora dueña de los productos, obtiene su `porcentaje_comision` personalizado y genera el registro `LiquidacionComercializadora` de forma desacoplada para mantener la trazabilidad contable.

Pregunta 3 (Toma de Auto-Pedidos): ¿Cómo se administra un pedido generado directamente por el rol de Tienda?

Respuesta: Cuando un propietario de tienda genera su pedido, este se crea dejando el campo `vendedor` nulo. A través de consultas centralizadas, este pedido cae en una "bandeja compartida" visualizada por todos los Vendedores; el primero que procese o administre el pedido asume el rol del `vendedor_id` y de las comisiones.

3.1. Diagrama de componentes

El sistema se compone de:

- Componente de Interfaz de Usuario (Django Templates): Sistema de renderizado de pantallas con HTML/CSS mediante vistas y formularios acoplados directamente al backend.
- Componente Core de Negocios (Django Models/Views): Módulo central de toda la aplicación que unifica la lógica relacional con base en roles y operaciones CRUD (Create, Read, Update, Delete).
- Componente de Persistencia (PostgreSQL): Motor de bases de datos para el almacenamiento seguro de relaciones transaccionales, control de stock y usuarios.

3.2. Arquitectura lógica y física (propuesta)

Arquitectura Lógica: Modelo Cliente-Servidor (Aplicación web monolítica basada en renderizado del lado del servidor - SSR - gestionada íntegramente por Django y su patrón MVT).

Arquitectura Física:

- Capa Cliente: Navegadores web en dispositivos móviles y computadoras de escritorio visualizando HTML enriquecido.
- Capa de Presentación / Proxy: Servidor Nginx que actúa como proxy inverso y sirve los recursos estáticos del sistema de plantillas.
- Capa de Aplicación: Contenedores Docker ejecutando Gunicorn para levantar y procesar la aplicación principal de Django de forma estable.
- Capa de Datos: Servidor de base de datos relacional PostgreSQL con protección ante concurrencia transaccional.

3.3. Prototipo de pantallas no funcional

Con el propósito de validar la experiencia de usuario y visualizar el alcance funcional de la plataforma antes de su implementación definitiva, se desarrolló un conjunto de prototipos no funcionales. Estas interfaces permiten representar los principales módulos del sistema y los flujos de interacción de los distintos tipos de usuarios.

- **Página de Inicio:** Pantalla principal de presentación de la plataforma, donde se describen los beneficios para comercializadoras, vendedores y micromercados, además de proporcionar acceso a las opciones de inicio de sesión y registro.

- **Pantalla de Inicio de Sesión:** Permite la autenticación segura de los usuarios registrados (Vendedores, Comercializadoras, Tiendas).
- **Pantalla de Registro:** Facilita el registro de nuevos usuarios bajo los tres roles habilitados, solicitando la información necesaria para su creación inmediata en base de datos.
- **Dashboard (Panel de Control Principal):** Vistas específicas para cada rol. El Comercio ve sus productos; el Vendedor analiza sus pedidos y comisiones; y la Tienda consulta su inventario local y estado de reabastecimiento.
- **Catálogo de Productos:** Presenta los productos disponibles agrupados por categoría comercial con visibilidad de precios y reglas de inventario.
- **Gestión de Inventario (B2B y B2C):** Permite al comercio mayorista abastecer su bodega principal, mientras a la tienda local le permite controlar su stock en percha.
- **Registro de Pedido:** Interfaz unificada usada tanto por Vendedores de ruta como por dueños de Tienda para generar pedidos calculando subtotales automáticamente.
- **Seguimiento y Transición de Pedidos:** Módulo destinado a consultar y avanzar el estado de pedidos. Permite inyectar inventario automáticamente hacia las tiendas una vez que son 'ENTREGADOS'.
- **Panel de Comisiones y Recompensas:** Tablero para que los vendedores consulten el efecto financiero de cada venta generada.

Cabe destacar que el sistema actualmente ya no es un prototipo y se encuentra en etapa funcional. La estructura visual renderizada hereda la maquetación definida en el sistema de plantillas y CSS base provistos en los componentes de Django.